

Lecture 8 – SQL Part 2

CPE241 – Database Systems

10 March 2026

Dr. Jaturon Harnsomburana

Dr. Piyanit Ua-areemitr

Department of Computer Engineering

King Mongkut's University of Technology Thonburi



Recap

- DDL: CREATE, DROP, ALTER, TRUNCATE, COMMENT, RENAME
- DML: INSERT, UPDATE, DELETE, LOCK, CALL, EXPLAIN PLAN
- DQL: SELECT
 - DISTINCT
 - WHERE Clause
 - ORDER BY
 - Build-in aggregate functions
 - JOIN

Check Point

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

STUDENT

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

Query Result

sid	name	cid	gpa
53650	Smith	Topology112	3.8
53688	Smith	NULL	3.2
53666	Jones	Jazz101	3.4
53666	Jones	Reggae203	3.4
53666	Jones	History105	3.4

SELECT _____
FROM _____
ON _____
ORDER BY _____

Example database

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Example database

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Query

Retrieve the birth date and address of the employee(s)
whose name is 'John B Smith'.

```
SELECT Bdate, Address  
FROM EMPLOYEE  
WHERE Fname = 'John' AND Minit = 'B' AND Lname = 'Smith';
```

Result: 1 row

<u>Bdate</u>	<u>Address</u>
1965-01-09	731Fondren, Houston, TX

JOIN

Retrieve the name and address of all employees who work for the 'Research' department.

```
SELECT Fname, Lname, Address  
FROM EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber  
WHERE Dname = 'Research';
```

Result: 4 rows

```
SELECT Fname, Lname, Address  
FROM EMPLOYEE, DEPARTMENT  
WHERE Dnumber = Dno AND Dname = 'Research';
```

Result: 4 rows

<u>Fname</u>	<u>Lname</u>	<u>Address</u>
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
Joyce	English	5631 Rice, Houston, TX

Unspecified WHERE Clause

Select all EMPLOYEE Ssns

```
SELECT Ssn  
FROM EMPLOYEE;  
Result: 8 rows
```

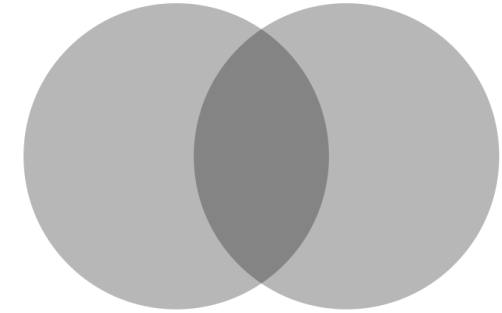
Select all combinations of EMPLOYEE Ssn
and DEPARTMENT Dname

```
SELECT Ssn, Dname  
FROM EMPLOYEE, DEPARTMENT;  
Result: 24 rows
```

Ssn	Dname
123456789	Research
123456789	Administration
123456789	Headquarters
333445555	Research
...	...

Note: CROSS PRODUCT - all possible row combinations

Tables as Sets



- UNION, EXCEPT (difference), INTERSECT
- Multi-set operations: UNION ALL, EXCEPT ALL, INTERSECT ALL (Duplicates are not eliminated.)

Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```
(SELECT DISTINCT Pnumber
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith')
UNION
(SELECT DISTINCT Pnumber
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE Pnumber = Pno AND Essn = Ssn AND Lname = 'Smith');
```

Result: 2 rows

Pnumber
1
2

Arithmetic Operations

- Standard arithmetic operators (+, -, *, /) may be included as a part of **SELECT**

Show the resulting salaries if every employee working on the 'ProductX' project is given a 10 percent raise.

```
SELECT E.Fname, E.Lname, 1.1*E.Salary AS Increased_sal
FROM EMPLOYEE AS E,WORKS_ON AS W, PROJECT AS P
WHERE E.Ssn = W.Essn AND W.Pno = P.Pnumber AND P.Pname = 'ProductX'
```

Result: 2 rows

Fname	Lname	Increased_sal
John	Smith	33000
Joyce	English	27500

INSERT from Query results

```
INSERT INTO table_name (column1, column2, column3)  
VALUES (value1,value2,value3);
```

```
INSERT INTO WORKS_ON_INFO (Emp_name, Proj_name, Hours_per_week)  
SELECT E.Lname, P.Pname, W.Hours  
FROM PROJECT P,WORKS_ON W, EMPLOYEE E  
WHERE P.Pnumber = W.Pno AND W.Essn = E.Ssn;
```

UPDATE with Query results

```
UPDATE Project  
SET Plocation = 'Bellaire', Dnum = 5  
WHERE Pnumber = 10;
```

Give all employees in the 'Research' department a 10% raise in salary.

```
UPDATE Employee  
SET Salary = Salary * 1.1  
WHERE Dno IN (  
SELECT Dnumber  
FROM DEPARTMENT  
WHERE Dname = 'Research');
```

NULL

- Meaning of **NULL**
 - Unknown value
 - Unavailable or withheld value
 - Not applicable attribute
- Each individual NULL value considered to be different from every other NULL value.
- SQL uses a three-valued logic: TRUE, FALSE, and UNKNOWN (like maybe).
- NULL = NULL comparison is avoided.

Table 5.1 Logical Connectives in Three-Valued Logic

(a)	AND	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	FALSE	UNKNOWN
	FALSE	FALSE	FALSE	FALSE
	UNKNOWN	UNKNOWN	FALSE	UNKNOWN
(b)	OR	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	TRUE	TRUE
	FALSE	TRUE	FALSE	UNKNOWN
	UNKNOWN	TRUE	UNKNOWN	UNKNOWN
(c)	NOT			
	TRUE	FALSE		
	FALSE	TRUE		
	UNKNOWN	UNKNOWN		

NULL

- SQL allows queries that check whether an attribute value is NULL
 - IS or IS NOT NULL

Retrieve the names of all employees who do not have supervisors.

```
SELECT Fname, Lname  
FROM EMPLOYEE  
WHERE Super_ssn IS NULL;
```

Result: 1 row

Nested Queries

- Use tuples of values in comparisons
 - Place them within parentheses

Select Essns of all employees who work the same combination (Pno, Hours) on some project that the employee whose Ssn = '123456789' works on.

```
SELECT DISTINCT Essn
FROM WORKS_ON
WHERE (Pno, Hours) IN (SELECT Pno, Hours
                       FROM WORKS_ON
                       WHERE Essn = '123456789');
```

Result: 2 rows

Pno	Hours
1	32.5
2	7.5

Nested Queries

- Use other comparisons operators to compare a single value v
 - = **ANY** (or = **SOME**) operator: return TRUE if the value v is equal to some value in the set V and is hence equivalent to IN
 - Other operators that can be combined with ANY (or SOME): >, >=, <, <=, <>
 - > **ALL**: value must exceed all values from nested query

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > ALL (SELECT Salary
                    FROM EMPLOYEE
                    WHERE Dno = 5);
```

Result: 2 rows

Salary
30000
40000
38000
25000

Nested Queries

- Avoid potential errors and ambiguities
 - Create tuple variables (aliases) for all tables referenced in SQL query

Retrieve the name of each employee who has a dependent with the same first name and is the same sex as the employee.

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE E.Ssn IN (SELECT Essn
                FROM DEPENDENT AS D
                WHERE E.Fname = D.Dependent_name AND E.Sex = D.Sex);
```

Result: 0 row

Nested Queries

- Query that are nested using the = or **IN** comparison operator can be collapsed into one single block.

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE E.Ssn IN (SELECT Essn
                FROM DEPENDENT AS D
                WHERE E.Fname = D.Dependent_name AND E.Sex = D.Sex);
```

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E, DEPENDENT AS D
WHERE E.Ssn = D.Essn AND E.Sex = D.Sex AND E.Fname = D.Dependent_name;
```

Nested Queries

Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```
SELECT DISTINCT Pnumber
FROM PROJECT
WHERE Pnumber IN (SELECT Pnumber
                   FROM PROJECT, DEPARTMENT, EMPLOYEE
                   WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith')

OR

Pnumber IN (SELECT Pno
            FROM WORKS_ON, EMPLOYEE
            WHERE Essn = Ssn AND Lname = 'Smith');
```

EXIST

- EXISTS - check whether the result of a correlated nested query is empty or not. They are Boolean functions.

List the names of managers who have at least one dependent.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE EXISTS (SELECT *
              FROM DEPENDENT
              WHERE Ssn = Essn)

AND

EXISTS (SELECT *
       FROM DEPARTMENT
       WHERE Ssn = Mgr_Ssn);
```

NOT EXIST

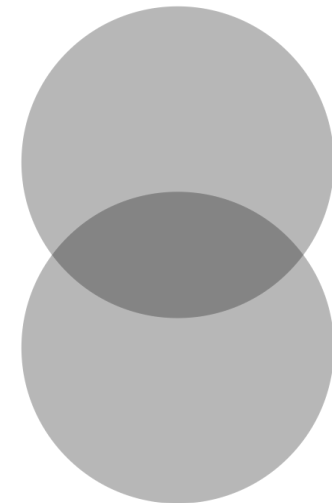
List first and last name of each employee who works on all the projects controlled by department number 5.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE NOT EXISTS ((SELECT Pnumber
                    FROM PROJECT
                    WHERE Dnum = 5)

                  EXCEPT (SELECT Pno
                             FROM WORKS_ON
                             WHERE Ssn = Essn));
```

Empty set means the employee works on all the projects.

Projects controlled by department number 5.



Projects that the employee works on.

Aggregate Functions - Grouping

- Partition relation into subsets of tuples
 - Based on **grouping attribute(s)**
 - **GROUP BY** clause - specify grouping attributes
 - Apply function to each such group independently

Aggregate Functions - Grouping

For each department, retrieve the department number, the number of employees in the department, and their average salary.

```
SELECT Dno, COUNT(*), AVG(Salary)
FROM EMPLOYEE
GROUP BY Dno;
```

Result: 3 rows

Dno	COUNT(*)	AVG(Salary)
5	4	33250
4	3	31000
1	1	55000

For each project, retrieve the project number, the project name, and the number of employees who work on that project.

```
SELECT Pnumber, Pname, COUNT(*)
FROM PROJECT, WORK_ON
WHERE Pnumber = Pno
GROUP BY Pnumber, Pname;
```

Result: 6 rows

Pnumber	Pname	COUNT(*)
---------	-------	----------

Aggregate Functions - Having

- HAVING clause - provides a condition to select or reject an entire group
- Only the groups that satisfy the condition are retrieved in the result of the query.

For each project on which more than two employees work, retrieve the project number, the project name, and the number of employees who work on the project.

```
SELECT Pnumber, Pname, COUNT (*)  
FROM PROJECT, WORK_ON  
WHERE Pnumber = Pno  
GROUP BY Pnumber, Pname  
HAVING COUNT (*) > 2;
```


Aggregate Functions - Grouping

For each department that *has more than five employees*, retrieve the department number and the number of its employees who are making more than \$40,000.

```
SELECT Dnumber, COUNT(*)  
FROM DEPARTMENT, EMPLOYEE  
WHERE Dnumber = Dno AND Salary > 40000 AND (  
    SELECT Dno  
    FROM EMPLOYEE  
    GROUP BY Dno  
    HAVING COUNT(*) > 5);
```

CASE

- Used when a value can be different based on certain conditions.
- Can be used in any part of an SQL query where a value is expected
- Applicable when querying, inserting or updating tuples

An example shows that employees are receiving different raises in different departments.

```
UPDATE EMPLOYEE
```

```
SET Salary = CASE WHEN Dno = 5 THEN Salary + 2000  
                  WHEN Dno = 4 THEN Salary + 1500  
                  WHEN Dno = 1 THEN Salary + 3000;
```

Assertion in SQL

- CREATE ASSERTION

- Specify a query that selects any tuples that violate the desired condition.
- Use only in cases where it goes beyond a simple CHECK which applies to individual attributes and domains.

Constraint: the salary of an employee must not be greater than the salary of the manager of the department that the employee works for.

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK (NOT EXISTS (SELECT *
                    FROM EMPLOYEE E, EMPLOYEE M, DEPARTMENT D
                    WHERE E.Dno = D.Dnumber AND D.Mgr_ssn = M.Ssn AND
                          E.Salary > M.Salary));
```

Trigger in SQL

- CREATE TRIGGER

- Specify automatic actions that database system will perform when certain events and conditions occur.
- Used to monitor the database.
- Typical trigger has three components: event(s), condition, action.

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF Salary, Supervisor_ssn ON EMPLOYEE
FOR EACH ROW
WHEN (NEW.Salary > (SELECT Salary
                    FROM EMPLOYEE M, DEPARTMENT D
                    WHERE Ssn = NEW.Supervisor_ssn))
INFORM_SUPERVISOR (NEW.Supervisor.Ssn, NEW.Ssn);
```

View in SQL

- CREATE VIEW

- Single table derived from other tables called the defining tables.
- Considered to be a virtual table that is not necessarily populated.
- Give table name, list of attribute names, and a query to specify the contents of the view.
- Attributes retain names from base tables.

```
CREATE VIEW WORKS_ON1
AS SELECT Fname, Lname, Pname, Hours
FROM EMPLOYEE, PROJECT, WORKS_ON
WHERE Ssn = Essn AND Pno = Pnumber
```

WORKS_ON1

Fname	Lname	Pname	Hours
-------	-------	-------	-------

View in SQL

- Attributes are assigned names

```
CREATE VIEW DEPT_INFO (Dept_name, No_of_emps, Total_sal)
AS SELECT Dname, COUNT (*), SUM (Salary)
FROM DEPARTMENT, EMPLOYEE
WHERE Dnumber = Dno
GROUP BY Dname
```

DEPT_INFO

Dept_name	No_of_emps	Total_sal
-----------	------------	-----------

View in SQL

- Once a view is defined, SQL queries can use the view relation in the FROM clause.
- View is always up-to-date (by the DBMS not the user).
- DROP VIEW - to dispose of a view.

Wrap up

- Set operator
- NULL
- Nested queries
- Grouping
- Assertion/ Trigger/ View

```
SELECT <attribute and function list>  
FROM <table list>  
[WHERE <condition>]  
[GROUP BY <grouping attribute(s)>]  
[HAVING <group condition>]  
[ORDER BY < attribute list>];
```